Rapport Projet Informatique

Jeremy RAKOTONIRINA, Mohamed Amine BERRAHMA, Eduardo COBOS FERNANDEZ

Sommaire

1. État des lieux
2. Architecture du projet
3. Evaluation d’un half-adder sur deux entiers
4. Profondeur et nombre de portes d’un half-adder
5. Code de Hamming

1. Etat des lieux

Aucun bonus n'a été implémenté

TD1: Exo 1-12: code ok, test ok

TD2: Exo 1-5: code ok, test ok

TD3: Exo 1-9: code ok, test ok
Exo 10: code ok, test ok. Mais on n'utilise pas la gestion des indices par un dictionnaire. On suppose qu' à chaque nouveau nœud construit, son indice est choisi de manière à ce que les indices de tous les nœuds soient toujours ordonnés (0,1,2,3...).

TD4 : Exo 1-3 : code ok, test ok

TD5 : Exo 1-5 : code ok, test ok

TD6: Exo 1-5 : code ok, test ok

TD7 : Exo 1-3 : code ok, test ok

TD8: Exo 1-4: code ok, test ok

TD9: Exo 1-4: code ok, test ok,

TD10: Exo 1-3: code ok test ok,

TD11: exo1 -5 : code ok(seulement affichage de evaluate pas assez simplifié)
test ok(en majorité basé sur des affichage d'additionneurs)

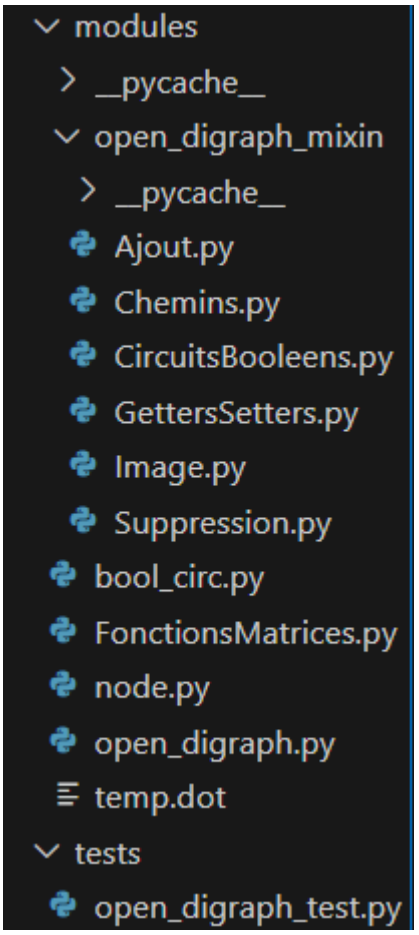
TD12: Exo 1: code ok, test ok

Exo 2: code fait partiellement. Nous avons réussi à implémenter les règles d'écriture mais pas la méthode qui applique ces règles tant que c'est possible.

Exo 3: code fait, test ok.

Architecture du projet

Voici l'arborescence de notre projet:



-Le dossier *modules* contient:

•*bool_circ.py*: importe le fichier *open_digraph*. Il contient la classe bool_circ et ses méthodes.

•*FonctionsMatrices.py*: importe la bibliothèque random. Le fichier contient les fonctions sur les matrices, plus particulièrement pour la génération.

•*node.py*: Le fichier contient la classe node et ses méthodes.

•*open_digraph.py*: importe les bibliothèques random, sys, os, matplotlib.colors. Il contient également les fonctions des fichiers *FonctionsMatrice.py* et *node.py* et ses fichiers de mixin.

Le dossier *open_digraph_mixin* contient:
•*Ajout.py* qui contient les méthodes d'ajout. importe le fichier node et les bibliothèques sys et os.
•*Chemins.py* qui contient les méthodes en rapport avec la profondeur, chemins…
•*CircuitsBooleens.py* qui contient les méthodes en rapport avec les circuits booléens. importe le fichier node.
•G*ettersSetters.py* qui contient les getters et les setters. importe le fichier node et les bibliothèques sys et os.
•*Image.py* qui contient les méthodes pour afficher le graphe graphiquement. importe le fichier node et les bibliothèques sys et os.
•*Suppression.py* qui contient les méthodes de suppression.

-Le dossier *tests* contient:
•*open_digraph_test.py* qui contient nos lignes de tests

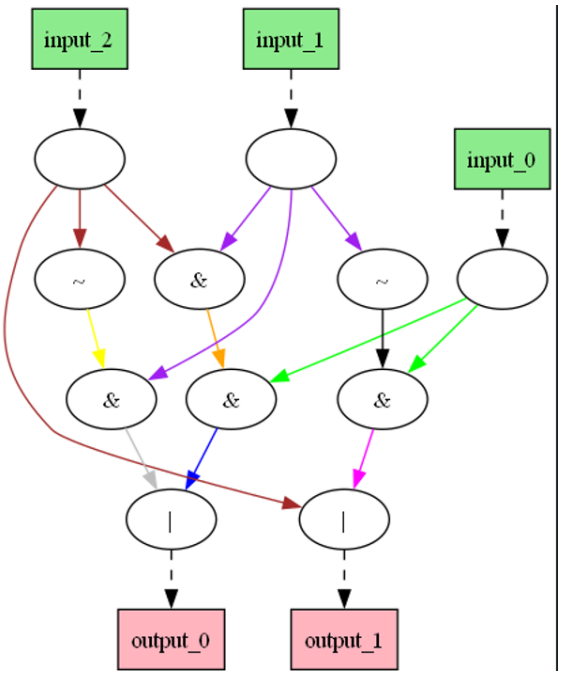
Voici quelques exemples de tests effectués:

```
Test random_int_matrix:
[0, 2, 1, 3, 2]
[3, 0, 3, 2, 2]
[0, 2, 0, 2, 1]
[0, 1, 3, 0, 0]
[0, 2, 1, 2, 0]

Test random_symetric_int_matrix:
[0, 0, 3, 3, 0]
[0, 0, 1, 1, 1]
[3, 1, 0, 1, 3]
[3, 1, 1, 0, 2]
[0, 1, 3, 2, 0]

Test random_oriented_int_matrix:
[0, 0, 0, 0, 0]
[1, 0, 3, 0, 1]
[2, 0, 0, 0, 0]
[0, 0, 3, 0, 1]
[1, 0, 0, 0, 0]

Test random_triangular_int_matrix:
[0, 0, 3, 0, 3]
[0, 0, 3, 0, 2]
[0, 0, 0, 1, 1]
[0, 0, 0, 0, 2]
[0, 0, 0, 0, 0]
```

Test dans la console des fonctions génératrices de matrice

```
open_digraph(
  Inputs: [4, 5],
  Outputs: [6, 7],
  Nodes:
  Node 0 (v0):
    Parents: {0: 2, 1: 1, 2: 1, 3: 2, 5: 1}
    Children: {0: 2, 1: 1, 3: 2}
  Node 1 (v1):
    Parents: {0: 1, 1: 2, 4: 1}
    Children: {0: 1, 1: 2, 2: 2, 3: 1}
  Node 2 (v2):
    Parents: {1: 2, 2: 1}
    Children: {0: 1, 2: 1, 6: 1}
  Node 3 (v3):
    Parents: {0: 2, 1: 1}
    Children: {0: 2, 7: 1}
  Node 4 ():
    Parents: {}
    Children: {1: 1}
  Node 5 ():
    Parents: {}
    Children: {0: 1}
  Node 6 ():
    Parents: {2: 1}
    Children: {}
  Node 7 ():
    Parents: {3: 1}
    Children: {}
)
[2, 1, 0, 2, 0, 0, 0, 0]
[1, 2, 2, 1, 0, 0, 0, 0]
[1, 0, 1, 0, 0, 0, 1, 0]
[2, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
```

Test dans la console de la fonction qui renvoie la matrice d'adjacence d'un graphe



Test de la fonction parse_parentheses sur les chaînes "((x0)&((x1)&(x2)))|((x1)&(~(x2)))" et "((x0)&(~(x1)))|(x2)" combinées

Nous avons pris soin de bien tester chacune des méthodes du projet avec près de 1500 lignes de code.

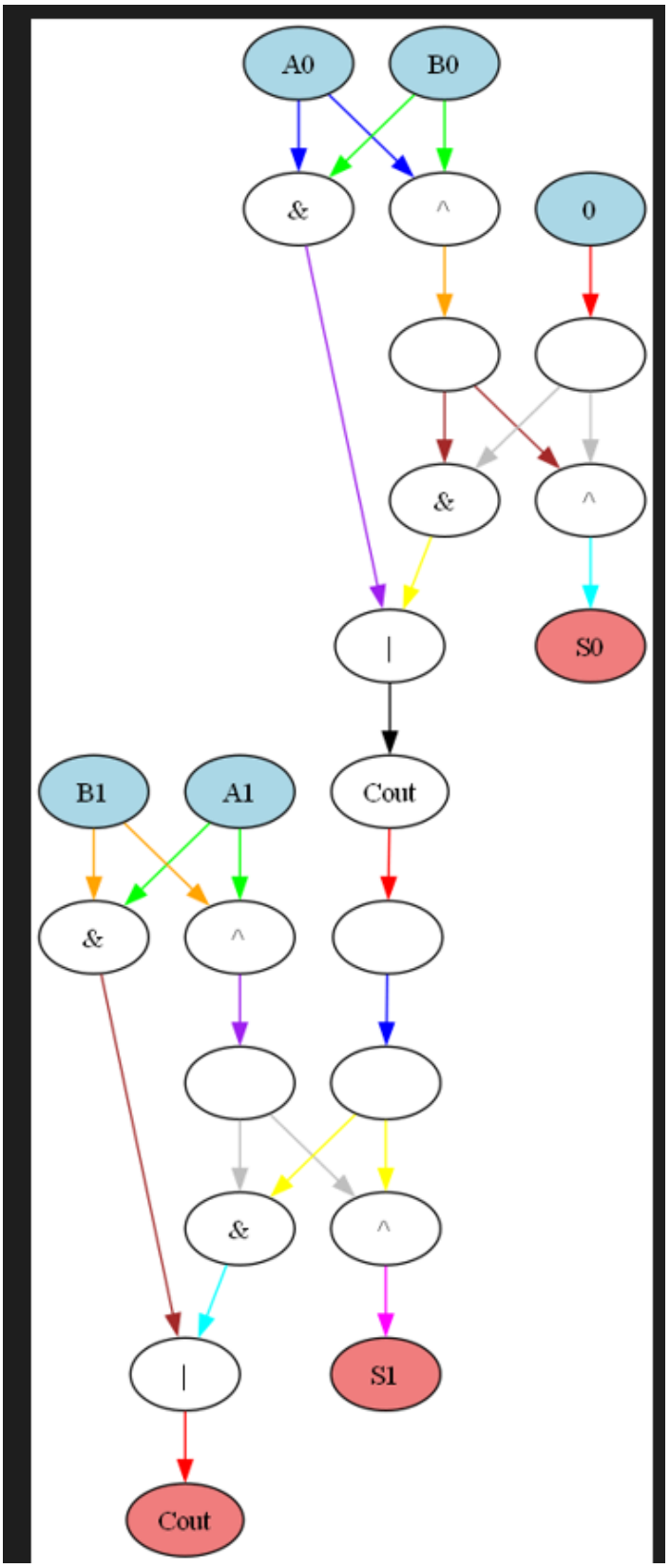
2. Evaluation d'un half_adder sur 2 entier :

Pour que cela soit simple nous allons faire la somme de 2 entiers très simples qui sont 1 et 2, (1+2=3)
1 en binaire s'écrit : 01 sur 2 bits (registre A)
2 en binaire s'écrit : 10 sur 2 bits (registre B)

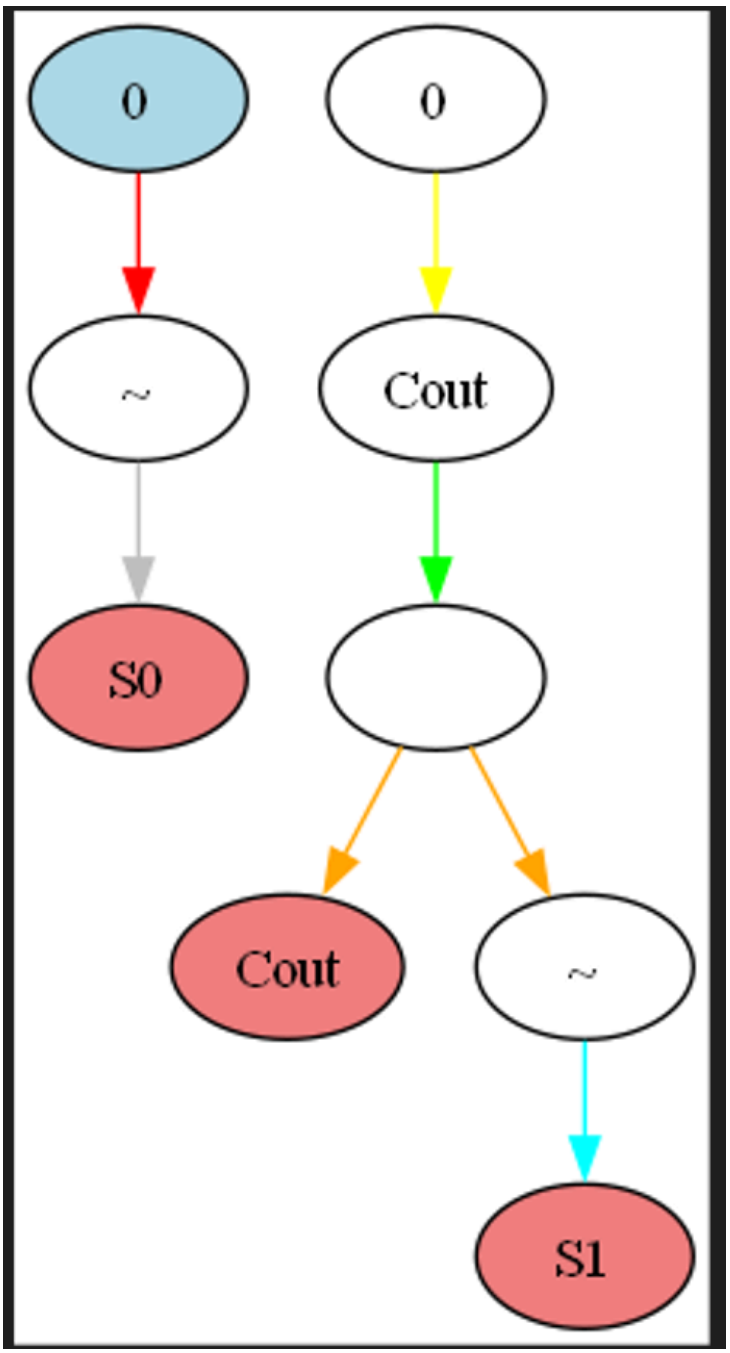
donc on va travailler sur un half adder(1) avec A0 = A, A1 = 0 et B0 = 0, B1=1. Voilà ce qu'on fait dans ce test si :

```
#test pour somme de 1+2 = 3
c = bool_circ.half_adder(1)
c.display()
c.evaluate({'A0':'1', 'B0': '0', 'A1':'0', 'B1': '1'})
c.display()
```

Le premier appel à display() lance l'affichage du circuit:



Ensuite, l'appel à evaluate() renvoie cet affichage :



Dans ce circuit résultat on voit clairement que not(0) vaut 1 d'ou S0 = 1 et aussi que copie(0) = 0 donc Cout = 0 et que not(copie(0)) = 1 d'où S1 = 1.
Donc notre résultat en binaire est bien : 011 ce qui correspond à 3.
L'évaluation du half adder marche aussi bien sur une somme d'autres entiers à registre plus élevé.

Cependant l'affichage pourrait être plus simplifié mais cela est dû à la fonction evaluate qui a été très dure à implémenter. En effet on a mis

plusieurs jours pour l'implémenter à elle seule, et on s'est aidé d'Internet pour cette fonction.

3. Profondeur et nombre de portes d'un half-adder:

Dans notre projet, notre half_adder(n) est construit à partir d'un adder(n) , qui additionne n+1 bits. Le half_adder(n) est obtenu en prenant cet adder(n) et en fixant sa retenue d'entrée à '0'.
Pour le adder 0 on a un total de 5 portes logiques : 2 porte '^', 2 porte '&' , et une porte '|'
Et pour ce half_adder(0), la profondeur est de 5. Pour la longueur du chemin le plus proche , on traverse 2 portes pour arriver à une sortie, donc 2.

-Généralisation au cas d'un half_adder(n) :

Le half_adder(n) est composé de 2^n 'half_adder(0) et on ajoute a chaque fois (2^n -1) portes donc le nombre de porte théorique est de
p(n) = 7*(2^n) + (2^n -1)

Pour ce qui est du plus court chemin théorique, il reste généralement déterminé par le premier bit et le chemin direct vers la sortie S0 donc la longueur = 2.

Pour ce qui est de la profondeur c'est un peu plus compliqué :

Cela dépend beaucoup de la manière dont les blocs de adder(0) sont connectés entre eux. Comme notre structure est 2^n(half_adder(0)), la retenue de sortie du half_adder(0) a une profondeur de 5. Ensuite pour atteindre la retenue de sortie de l'étage d'après il faut ajouter 5 portes donc '5+5'. Pour avoir la retenue de sortie du ième étage , on ajoute (n^2)*5 a la profondeur initiale a chaque fois d'où on obtient une formule finale théorique de **5* 2^n +1** pour n>= 1 car à chaque fois on a un nœud Cout en plus.

-Vérification expérimentale avec notre half_adder :

Nombre de portes :

•pour n = 0 :
nombre de portes théoriques : 7*(2^0) +0 = 7 et le nombre réellement mesuré est 7 donc c'est conforme,

•pour n = 1 le nombre de portes théoriques : 7*(2^1) + 1 = 15 et le nombre réellement mesuré est 15 donc c’est conforme

•pour n = 2 le nombre de portes théoriques : 7*(2^2) + 5 = 31 et le nombre réellement mesuré est 31 donc c’est conforme

•pour n = 3 le nombre de portes théoriques : 7*(2^3) + 7 = 56 et le nombre réellement mesuré est 63 donc c’est conforme

La longueur du plus court chemin est de 2 a chaque fois

•pour n = 1 la profondeur théorique est de 5+2^1 +1 = 11, et la profondeur expérimentale calculé à l'aide de notre fonction profondeur graphe donne bien 11 ce qui est conforme

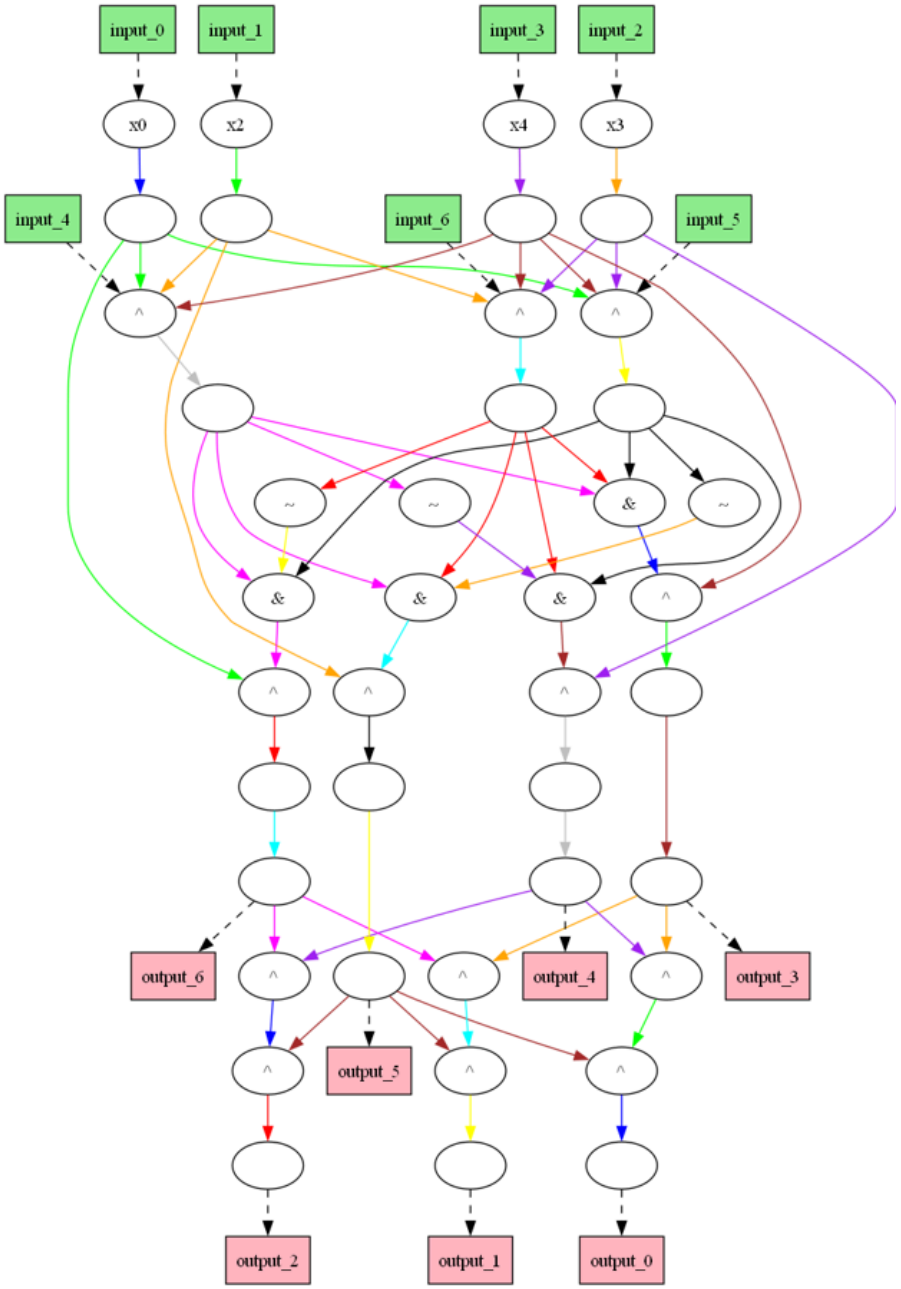
•pour n = 2 la profondeur théorique est de 5+2^2 +1 = 21, et la profondeur expérimentale calculé à l'aide de notre fonction profondeur graphe donne bien 21 ce qui est conforme

•pour n = 3 la profondeur théorique est de 5+2^3 +1 = 41, et la profondeur expérimentale calculé à l'aide de notre fonction profondeur graphe donne bien 41 ce qui est conforme

Voilà donc les formules théoriques pour la profondeur et le nombre de portes du half_adder.

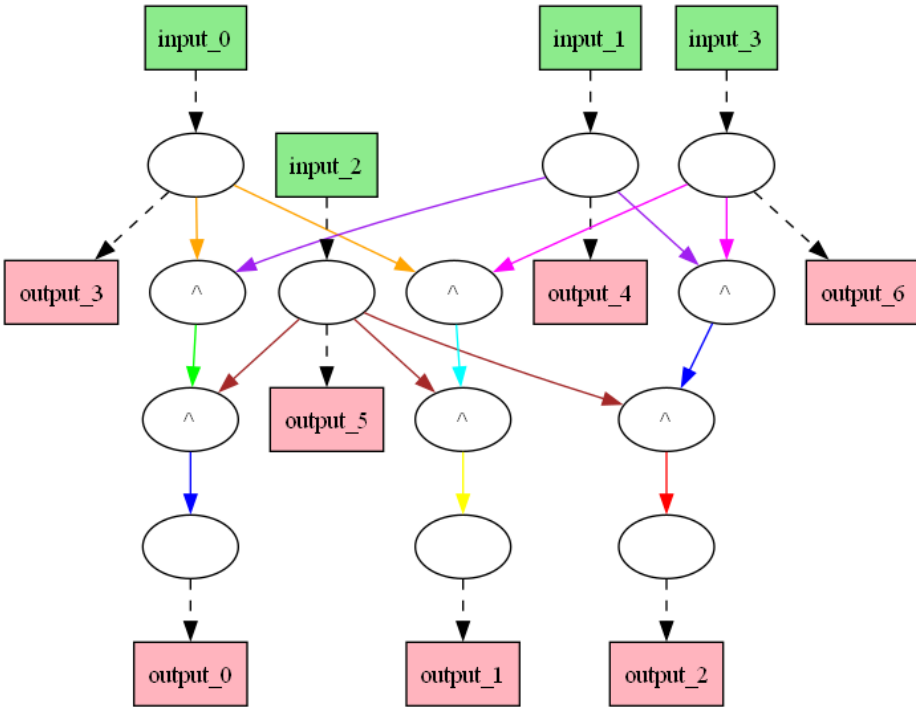
5. Code de Hamming

Cet exercice est particulièrement long car nous n'avons pas réussi à faire la fonction qui applique les méthodes de réécriture.
Pour vérifier la propriété principale du code de Hamming, nous avons dû faire une combinaison séquentielle de l'encodeur et du décodeur ce qui a donné:



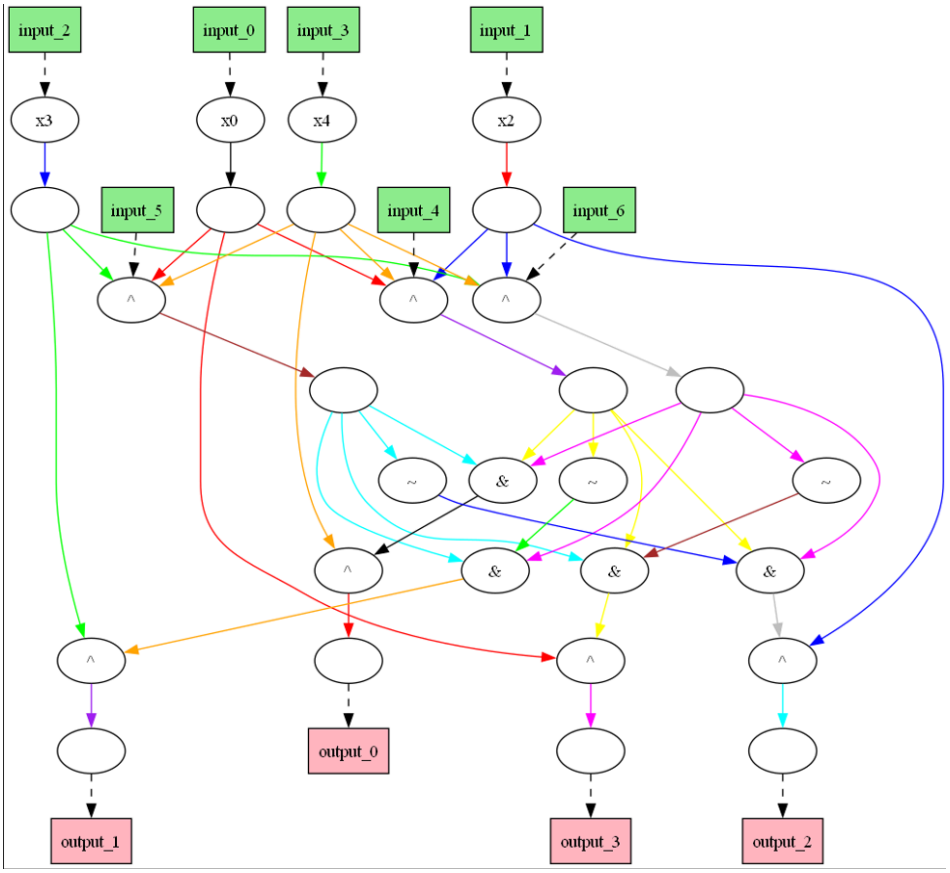
Ensuite, faire traverser une chaîne de 4 bits et vérifier que ça renvoyait bien l'identité. Nous l'avons fait à la main avec des bits aléatoires.

Ensuite, pour vérifier qu'une unique erreur sur les bits renvoyait toujours la même chose. On a encodé 4 bits,



Encodeur

Puis, on a pris 1 bit au hasard et on a changé de valeur. On l'a appliqué au décodeur, et ça renvoyait toujours le bon résultat.



Decodeur

Pour vérifier la dernière propriété qui est que 2 erreurs peuvent ne pas retrouver le message d'origine on a fait des essais successifs (2 ou 3) avant d'y arriver. On a pris 2 positions i et j distinctes et on a changé les valeurs des bits i et j.